

Fig. 3. Design challenges. (a) Throughput comparison of GustANN and the *GPU direct-access* solution (*Direct* in the figure). (b) The SSD bandwidth comparison of BaM (a recent GPU direct-access solution), and the theoretical maximum (*Ideal* in the figure). *PCIe* is the maximum PCIe transfer bandwidth of the GPU. (c) The distribution of the SSDs' read latency. Each line indicates an SSD.

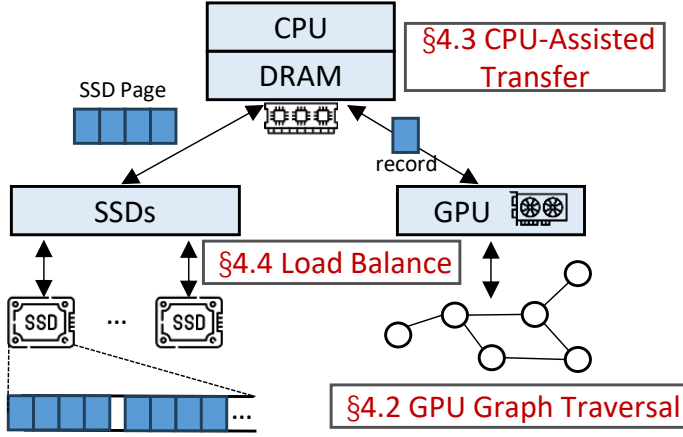


Fig. 4. GustANN architecture.

4 Design

4.1 Overview

GUSTANN uses a *GPU-centric, CPU-assisted architecture* to achieve high-throughput and cost-effective ANNS. Its overall architecture is shown in Figure 4.

4.1.1 Data Placement. The data placement on disks of GUSTANN is similar to DiskANN [30] (§2.2), which stores vectors and neighbors on disks. In addition, GUSTANN uses multiple SSDs, and the data is stored in a RAID 0 scheme. To tackle the load balance issue (Challenge 3), we leverage a pivot search (§4.4) and store a pivot graph in GPU memory. Also, most data structures (e.g., compressed vectors) are moved from DRAM to GPU memory.

The main usage of DRAM in GUSTANN is the bouncing buffers in the CPU-assisted transfer to enable selective transfer from SSD to GPU. Instead of GPU direct-access, this method can mitigate the bottleneck of the PCIe bandwidth for the GPU (§4.3). GUSTANN mainly uses DRAM for its bandwidth, and its capacity demand is much smaller compared to other DRAM-based billion-scale ANNS systems, thus reducing costs for servers (§5.5).

4.1.2 Search Process: Single Vector. GUSTANN leverages GPU for graph traversal tasks to increase the bandwidth utilization of SSDs, ensuring search throughput.

Initialization. GUSTANN initializes a query similarly to DiskANN. In addition, the pivot search for load balancing is performed (more detail in §4.4) to decide the entry node of the main search.

Search. The search process is led by the CPU, which coordinates reading on SSDs and searching on the GPU. To overcome the inbound bandwidth limit of the GPU, we leverage the CPU to assist in initiating the SSD read process (§4.3).

The searching process follows a similar scheme to Algorithm 1 (§2.2), except that most computation is moved to the GPU. It consists of repeated iterations until all vectors in the candidate set are explored. In each iteration, there are two main phases:

- (1) *SSD phase*, where we fetch the corresponding SSD page of a selected exploring node u from SSD to DRAM, and then transfer the desired record in the page to GPU (§4.3).
- (2) *GPU phase*, which performs distance calculations and data structure updates on GPU. The GPU phase also selects the exploring node for the next SSD phase (§4.2).

4.1.3 Search Process: Multiple Vectors. To support high throughput ANNS, GUSTANN puts queries into batches to further exploit the computing power of GPU and read bandwidth of SSDs. Note that we optimize the memory usage of each query (§4.2) to ensure a large batch size and thus a high concurrency level.

However, batched queries still cannot fully utilize GPU and SSDs. The graph traversal is sequential, with strong dependencies between stages, such as the exploring node and the record from SSD. As a result, two phases cannot run simultaneously, and half of the hardware resources are wasted.

To further extract hardware potentials, GUSTANN introduces *inter-query pipelining*. Instead of a single large batch for all queries, we split them into several mini-batches. These batches are independent of other batches, so both stages can be filled with different mini-batches. Therefore, all hardware resources can be utilized.

GUSTANN does not require strong CPU for assistance. During the search process, the main task of the CPU is only to schedule and dispatch different batches. Therefore, there is no need for CPU with strong computing power or special hardware features (e.g., advanced SIMD instructions). In addition, CPU with a low number of cores (e.g., 8 cores) can also serve GUSTANN efficiently. In our configuration (§5.1), we only need 2 threads for sending requests to SSDs and the GPU, and 6 threads for SSD I/O queue management (each sending ~2M IOPS requests to a designated SSD).

4.2 Memory-Efficient Graph Traversal Kernel

In this section, we address the high memory consumption issue in §3.3 (Challenge 1) by redesigning the GPU kernels for graph ANNS. We first analyze the memory footprint of the ANNS searching procedure. Then, we point out the limitations of the current implementation and our observations. Finally, we present our search algorithm to optimize the memory usage.

4.2.1 Memory Footprint Analysis. We analyze the memory footprint of one iteration in the search process shown in Algorithm 1 (procedure with full-precision vectors is omitted for its negligible time and memory overhead; Only per-query data structures are included below):

- (1) Read all neighbor IDs from the *record* (on SSD).
- (2) Calculate the PQ distance with the *PQ lookup table*.
- (3) Insert the (neighbor ID, PQ distance) pair into the *candidate set*, which should be (partially) sorted.

Note that to keep the correctness of the algorithm, all element IDs in the candidate set should be distinct. For the CPU-based algorithm, it is natural to maintain an auxiliary data structure (denoted as the *visited table*), such as a hash table or bloom filter [30, 48], to identify whether a node has been calculated *before* step (2), and thus reduce redundant computation.

The visited table holds up a large proportion of memory. This is because, for each iteration, all R neighbors of the current exploring node should be inserted into the visited table. As the search needs at least L iterations to explore all nodes in the candidate set, space for at least $L \times R$ nodes should be reserved. For example, when $L = 300$ and $R = 128$, and each neighbor uses 4 bytes, the memory consumption of the visited table is at least 150 KB. In practice, such data structures need to be half-full to prevent performance degradation, meaning their real size should typically be doubled ($> 300\text{KB}$). In comparison, the candidate set only stores L nodes ($1 - 2\text{ KB}$). Other structures (i.e., the record and PQ lookup table) have fixed sizes ($\sim 40\text{ KB}$) throughout the search. In addition, as GUSTANN needs high concurrency (~ 40000 concurrent queries), $> 11.5\text{ GB}$ of memory is needed for that single data structure, which is a large memory consumption given the limited GPU memory space (40 GB in our setup) and the memory need of other global data structures (such as PQ code, which needs $> 30\text{ GB}$ of space for billion-scale datasets).

4.2.2 Sort First, Then Deduplicate. Memory-consuming visited table restricts the batching capability of GPU due to the limited GPU memory, thus further reducing the search throughput. To save space, some approach on GPU uses linear lookup and insertion [26]. However, the time consumption is usually unacceptable.

To save memory space while not incurring much time overhead, we adopt the following observation: **Redundant computation is acceptable on GPU**. For ANNS on the CPU, it is natural to eliminate the duplicate neighbors before distance calculation, so that there is no redundant computation for PQ distances. However, GPU threads are executed in the granularity of warps (32 threads each). Threads in the same warp always execute the same instruction or are waiting (even if they should execute other branches). Therefore, there is no need to eliminate the redundant computation in one thread, as it will be hidden by other threads.

Based on this observation, we first perform the distance calculation and sorting procedure, before we deduplicate the sorted array. As a sorted array can be deduplicated with a linear scan (detailed in §4.2.3), no extra data structures such as the visited table are needed. In addition, the warp structure of the GPU ensures that the redundant computation in distance calculation and sorting can be hidden by other threads, and does not cause much time overhead.

4.2.3 Algorithm. We present our search algorithm, which first sorts and then deduplicates. In the search, we organize the *candidate set* as a sorted array according to the PQ distance. Each item in the array contains the candidate node's ID and the PQ distance. In addition, the information of the explored node (see Algorithm 1) is stored as an extra bit for each item.

For each search iteration, we first read all neighbors for the current exploring node (SSD phase). Then we add these neighbors to the candidate set with the following steps (GPU phase), as shown in Figure 5(a):

PQ distance calculation. We first calculate the PQ distances of the current exploring node's neighbors (Step ❶). We use R (number of neighbors) GPU threads, each of which calculates one neighbor's PQ distance. By assigning distance calculation for each neighbor to one thread, the redundant computation of duplicated neighbors can be hidden by the computation of other neighbors due to the warp property of the GPU.

Sorting. After that, all neighbors are appended to the candidate set, and the candidate set is sorted according to the PQ distances (Step ❷). Sorting consists of two substeps. First, we sort all neighbors

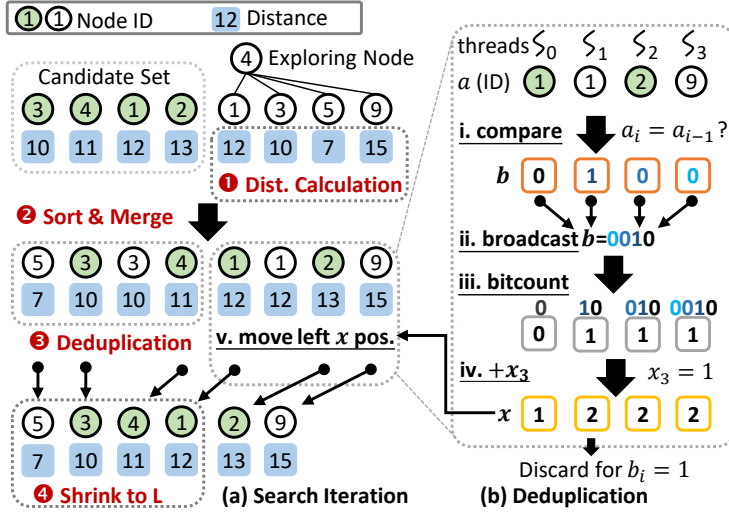


Fig. 5. The search algorithm on GPU. (a) One search iteration. (b) Parallel deduplication in the iteration. We show the round $k = 1$, which processes $a_4 \sim a_7$. For simplicity, we set $B = 4$ (the number of elements to be deduplicated in parallel) and $L = 4$ (size of the candidate set) in the figure.

retrieved from the last step. After that, we merge these neighbors with the remaining part of the candidate set (i.e., sorted nodes from previous iterations). We adopt parallel merge sort [25] for these two substeps. We also use R GPU threads in this step, so that the sorting procedure can be fused with the PQ distance calculation kernel, making better use of the high-bandwidth shared memory in the GPU.

Deduplication. After sorting, the candidate set is updated with new neighbors and sorted according to the distance, but duplicate items remain. To ensure the correctness of the algorithm, deduplication must be performed (Step ③), before selecting the first L elements as the new candidate set (Step ④). The deduplication of a sorted array is a traditional algorithm with a linear scan (for example, `std::unique` in C++ STL). However, a parallelized deduplication on the GPU is non-trivial, which needs to handle data dependency carefully.

We first present a non-parallelized version of the deduplication procedure. It is done with a linear scan of all node IDs in the candidate set, denoted as array a . During the scan, we maintain x_i , which indicates the number of duplicated elements from a_1 to a_i . We use the following formula to compute x_i :

$$\begin{cases} x_i = x_{i-1} + \delta(a_i, a_{i-1}), & i \geq 1 \\ x_0 = 0, \end{cases} \quad (1)$$

where $\delta(m, n)$ returns 1 if $m = n$ and 0 otherwise. After that, we move a_i (and other elements for that node) to a_{i-x_i} for all i satisfying $a_i \neq a_{i-1}$. This step removes all duplicates.

To utilize the parallel ability, we use $B = 32$ GPU threads (a warp) for the scan. However, the scan must be redesigned to fit the parallel scheme, as illustrated in Figure 5(b). We divide the scan into multiple rounds, with each round scanning B elements in a simultaneously. In the k -th round of scan, the i -th thread ($0 \leq i < B$) processes the $(Bk + i)$ -th element. It first calculates $b_i = \delta(a_{Bk+i}, a_{Bk+i-1})$ (Step i). After that, each thread broadcasts b_i to other threads, which can be done with the warp primitives of CUDA [37] (Step ii). The result can be represented as a B -bit

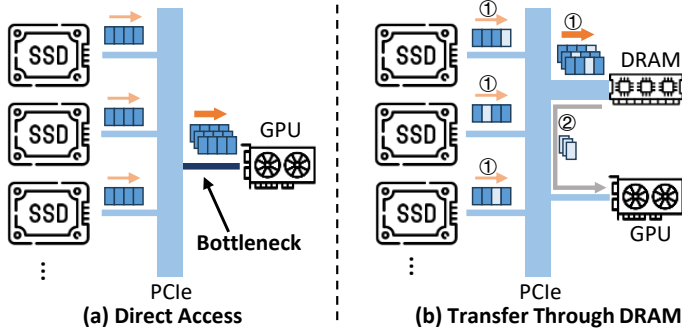


Fig. 6. Comparison of (a) GPU direct-access to SSD, which causes bandwidth bottleneck, and (b) first transfer the whole page to the bouncing buffer in DRAM, and then transfer the selected record to GPU.

integer $\mathbf{b}$, where the i -th bit of $\mathbf{b}$ is b_i . After synchronization, the i -th thread computes:

$$x_{Bk+i} = x_{Bk-1} + \text{bitcount}(\mathbf{b}_{0..i}) \quad (2)$$

where $\mathbf{b}_{0..i}$ is the first i bits in $\mathbf{b}$ (Step iii and iv). Note that x_{Bk-1} has been computed in the previous round, and we assume $x_{-1} = 0$ for $k = 0$ (first round). Next, each thread performs the data move mentioned above (Step v).

A further observation is that each duplication appears only twice in the list (once in the neighbor list and the other in the previous candidate set). As a result, only the first $\min(L + R, 2L)$ elements before deduplication may become the new candidate. Therefore, we only perform scans on these items.

Note that the exploring node for the next iteration can also be found during the scan, by selecting the first element that has not been explored.

4.3 CPU-Assisted Transfer

Bandwidth between SSDs and the GPU is asymmetric, which degrades SSD utilization, as shown in Figure 6(a). On the SSD side, as SSDs do not share PCIe lanes, their bandwidth can be aggregated to around 60 GB/s with 6 SSDs (our setup in §5.1) with PCIe 5.0 x4. However, on the GPU side, the bandwidth is limited to 24 GB/s with PCIe 4.0 x16 for NVIDIA A100. Therefore, GPU direct-access to SSDs (such as BaM [42], GDS [7]) cannot fully exploit the bandwidth potential of multiple SSDs.

To address this challenge, we find that only one record is used for each iteration, which typically accounts for less than 25% of the whole SSD page. During the GPU phase of the search, the GPU threads read one SSD-resident record, which is typically less than 1KB. For example, the vectors in the DEEP1B [52] dataset have 96 dimensions, and each dimension of the vector is a 32-bit floating number. With graph degree $R = 128$, the total record size would be $\text{sizeof(float)} \times 96 + \text{sizeof(int)} \times 128 = 896$ bytes. Meanwhile, SSDs have a read granularity of 4 KB, meaning less than 25% of the data may be used by the search algorithm.

This observation inspires us that we can selectively transfer data to the GPU. GPU direct-access cannot achieve selective transfer. Therefore, we leverage CPU to assist this transfer process, with CPU DRAM used as a bouncing buffer, as shown in Figure 6(b)

Step 1: The whole SSD page is transferred from SSDs to DRAM. As the transfer bandwidth from the PCIe root complex to DRAM is higher, the full transfer speed of SSDs can be achieved. In our 6-SSD setup, the bandwidth of 4 DRAM chips meets the need for the aggregated SSD bandwidth of ~ 60 GB/s. (§5.5.1 further discusses the DRAM usage.)

Step 2: Only the graph record of the exploring node is sent from DRAM to the GPU. Compared to GPU direct-access, this method eliminates the transfer of unneeded data to the GPU, thus reducing $> 75\%$ of the GPU inbound bandwidth demand. For example, a maximum of $60 \times 25\% = 15$ GB/s of GPU inbound bandwidth for the DEEP1B dataset is required, which is within the maximum bandwidth of the GPU.

Therefore, as GPU inbound bandwidth is no longer a bottleneck in the data transfer process, the bandwidth strength of multiple SSDs can be fully utilized, thus providing an opportunity for higher search throughput. In contrast, GPU direct-access methods only support reading in page granularity, which causes a waste of GPU inbound bandwidth.

Note that some datasets with high dimensions (e.g. OpenAI Embeddings [5] have 1536 dimensions) may exceed the SSD page size. However, these high-dimensional embeddings can be compressed to a smaller number of dimensions without losing much precision [23, 34, 51, 52]. Therefore, we can first reduce their dimensions before storing them.

4.4 Load Balance with Pivot Search

In this section, we first analyze the cause of load imbalance (Challenge 3). After that, we present our solution with a GPU-resident pivot graph.

4.4.1 Single Entry is not Efficient for Multi-SSDs. Many graph-based methods [21, 30, 39] adopt a single-entry design, where queries start the graph traversal from the same entry node.

However, such a single-entry design is inefficient as it causes a phenomenon we refer to as *micro-scale load imbalance*. This is because all queries only visit the nearby region around the entry node of the graph in the first few iterations, and only a few nodes are possible to be visited. For example, only 103 nodes for the first iteration, and 2783 nodes for the second iteration are visited in a billion-scale dataset [31]. Only nodes around 10^{-6} of the whole dataset are visited.

Such a small number of nodes are vulnerable to the load imbalance issue. For example, for a 10000-vector query set, while the median access count for the first iteration is 48, the maximum access count is 657 ($13.7\times$). Such a divergence in access counts results in varied performance of SSDs, especially in terms of tail latency, as shown in Figure 3(c). This imbalance is further magnified because all queries need to visit this region in the single-entry design.

4.4.2 Solution: GPU-resident Pivot Graph. Based on the analysis above, we argue that these queries should be scattered to *different regions* of the graph instead of the single region around the entry node. This method has two advantages: first, it mitigates load imbalance because each region now has fewer nodes, so the divergence will be less significant. In addition, as the target for the first few iterations in a single-entry graph-based search is to locate the top-1 nearest neighbor of the query [55], a region closer to the nearest neighbor of the query will result in better search speed.

To find a closer region, we introduce the *pivot search* technique: we randomly select *pivot vectors* from datasets (e.g., 0.1% of a billion-scale dataset) to represent different regions, and build a *GPU-resident pivot graph* G' on the GPU memory. For a query, we first perform a top-1 search on G' (the pivot search). The result (the nearest vector of the sampled set) will be used as the entry node of the SSD-resident graph search.

Pivot search has a similar search scheme to §4.2. Here, we mention some key differences: (1) To reduce the overall search overhead, we use a smaller *search length* (i.e., the length of the candidate set) for the pivot search (default value is 4). This is because pivot search does not need high recall for the nearest neighbor. (2) We use full-precision vectors instead of compressed PQ codes, as the pivot graph does not consume much memory. (3) We use the selection sort instead of the merge sort. This is because the search length is small, and the selection sort can be better parallelized under such circumstances.

It is worth mentioning that Starling [49] also uses a similar technique called the navigation graph to reduce the number of SSD I/O. However, GUSTANN differs from Starling in two aspects: (1) GUSTANN introduces the GPU-resident pivot graph to mitigate the load imbalance of SSDs, caused by the one-entry design. (2) GUSTANN uses GPU, instead of CPU, to perform the pivot search. Therefore, a different GPU-based search scheme is adopted, as shown above.

4.5 Discussion

4.5.1 Scalability of Data Size. In this section, we explore the possibility for scaling the data size by analyzing the storage usage of GUSTANN. GUSTANN mainly uses SSD and GPU memory, with a low DRAM usage (~ 200 MB for the bouncing buffers in the search).

On the SSD side, we store the vectors and the graph index as described in Figure 1. Billion-scale dataset (e.g., DEEP1B [52]) takes up ~ 950 GB of SSD space. Given that the capacity of modern SSDs can reach several TBs (e.g., 3.84 TB in our setup), the SSD will not be the bottleneck for scaling.

The GPU memory is mainly used for two purposes in GUSTANN. The first part is compressed vectors for distance approximation. Each compressed PQ vector is 32 bytes, and ~ 30 GB is required for the billion-scale dataset. To support larger datasets, recent quantization techniques such as LVQ [15], RabbitQ [22, 24] attempt to further reduce the memory consumption of compressed vectors (e.g., from 32 B to 16 B). Such methods could double the dataset size, and are orthogonal to GUSTANN.

The GPU memory is also used to manage the search state of the queries. GUSTANN achieves a low memory budget for the search (~ 40 KB per query) by making the deduplication procedure memory-efficient. This provides more space for larger datasets.

Another approach for scalability is to increase the GPU memory capacity. For example, with 80 GB GPU memory ($2\times$ of our experiment setup), GUSTANN could support larger datasets (i.e., 2 billion vectors) without performance loss.

Combining larger memory ($2\times$) and better quantization, GUSTANN could support 4 billion vectors with a single GPU.

4.5.2 Generalization of the Techniques. While GUSTANN targets at optimizing on-SSD ANNS with GPU, some ideas can be generalized to other problems:

- §4.2.3 provides a way to deduplicate with a low memory budget on the GPU. Other deduplication tasks on GPU could be done similarly (e.g., performing DISTINCT constraint). In the deduplication algorithm, two main techniques can be generalized: ❶ parallel predicates evaluation for on-GPU databases (Step **i** in Figure 5), and ❷ prefix sum on GPU with warp instructions (Step **ii** to **iv**).
- §4.3 introduces CPU-assisted selective transfer, which can be generalized to mitigate read amplification from SSD to GPU, potentially including on-SSD databases that utilize GPU for processing acceleration (e.g., when data processing on GPU is performed on a filtered subset of on-SSD data).

4.5.3 Limitation. To exploit the concurrent ability of GPU and SSD, GUSTANN trades off the search latency (~ 140 ms latency to reach the maximum throughput, see §5.6).

However, emerging applications, such as RAG, are not latency-critical because the latency of the whole process is mainly on the LLM side (e.g., prefill stage of LLM takes 100ms to 1s) [32, 38]. On the other hand, given the growing need for LLM and RAG applications, the ANNS in such scenarios are under much higher throughput demand, and we believe GUSTANN achieves a good throughput-latency tradeoff.

Some traditional ANNS applications, such as web search and recommendation systems, are typically considered latency-critical (e.g., 10 ms scale) [21, 30]. In these scenarios, GUSTANN may not achieve the maximum throughput due to the strict latency requirement.

5 Evaluation

In this section, we evaluate GUSTANN, seeking to answer the following questions:

- How does GUSTANN compare to other ANNS systems on both CPU and GPU for search throughput? (§5.2)
- How do designs of GUSTANN contribute to the search efficiency? (§5.3)
- Is GUSTANN sensitive to different workloads? (§5.4)
- How does GUSTANN compare to other ANNS systems in terms of cost-effectiveness? (§5.5)
- How does GUSTANN perform in terms of latency? (§5.6)

5.1 Experimental Setup

Basic configuration. We use a server with the following configuration for evaluation:

- **CPU:** 2× 28-core Intel Xeon Gold 5420+;
- **RAM:** 256 GB (8× 32 GB DDR5 4800MT/s);
- **SSD:** 6× Samsung PM1743 3.84 TB;
- **GPU:** NVIDIA A100-40GB;
- **OS:** Ubuntu 22.04 LTS with Linux kernel 6.2.0;
- **CUDA:** Version 12.3.

Datasets. We use 4 widely used public datasets for the evaluation:

- SIFT1B [31] is a dataset with 1 billion vectors of 128 `uint8` dimensions. The query set size is 10000.
- DEEP1B [52] is a dataset with 1 billion vectors of 96 `float` dimensions. The query set size is 10000.
- SIFT100M [31] is a subset of SIFT1B with 100 million vectors, and shares the same query set with SIFT1B.
- DEEP100M [52] is a subset of DEEP1B with 100 million vectors, and shares the same query set with DEEP1B.

Compared systems. We compare GUSTANN with two types of ANNS systems: (1) ANNS on CPU with SSD, and (2) ANNS on GPU with DRAM.

For ANNS on CPU with SSD, we use DiskANN, Starling, and SPANN. These systems store indexes in the SSD and use the CPU for computation. DiskANN [30] is a graph-based SSD-resident ANNS system on the CPU. Starling [49] improves DiskANN by reshuffling the indexing layout of SSDs and searching procedures. In addition, Starling uses a navigation graph to reduce the SSD I/O counts. SPANN [18] is a cluster-based ANNS system on the CPU. Cluster-based methods partition vectors into multiple clusters and search for the query in some selected clusters. SPANN leverages SSDs to store clustered vectors.

To store the vector index on the SSD, GUSTANN, DiskANN, and Starling have similar index structures and need ~65/~95 GB of storage space for SIFT100M/DEEP100M respectively. SPANN needs ~87/~235 GB for these two datasets. For the billion-scale dataset, the storage requirement is roughly 10× of the 100M-scale datasets (e.g., GUSTANN needs ~650/~950 GB for both datasets).

For ANNS on GPU with DRAM, we use BANG and RUMMY in the evaluation. BANG [48] is a recent graph-based ANNS system, and RUMMY [56] is a cluster-based ANNS system. They store indexes in DRAM and use the GPU for computation. BANG uses the DiskANN-style graph index